



Programma

Un programma C++ è una collezione di istruzioni (valide) del linguaggio incapsulate nel costrutto:

```
int main()  
  
    //dichiarazione di variabili  
  
    //istruzioni  
  
    return 0;  
}
```

Un programma in esecuzione esegue le istruzioni seguendo il flusso di calcolo determinato da sequenza, selezione e iterazione.

Il programma termina quando viene eseguita l'istruzione `return 0;`

Istruzioni

Assegnamento, I/O

```
/*Istruzioni*/  
a=5;  
b=2*a;  
c=a+b;  
  
//Istruzione stampa a video|  
cout << endl << "risultato in c: " << c << endl;  
  
/*fine programma*/  
return 0;
```



Blocco di istruzioni

Istruzioni racchiuse tra parentesi (C++: graffe) o altrimenti identificate (Python: indentazione, other **begin...end**).

```
{//Qui inizia un blocco di istruzioni  
  a = 3;  
  cout << endl << "Sono un blocco" << endl;  
  //...|  
}  
//qui finisce il blocco
```

Lettura e stampa dati

libreria <iostream>

- Per poter "dialogare" con un programma (fornire informazioni in ingresso/visualizzare risultati prodotti) è necessario usare due strumenti della libreria (leggi "collezione di strumenti") **<iostream>**.
- **cin >> nomevariabile:** legge da tastiera una sequenza di caratteri, li "interpreta" e registra quanto interpretato nella variabile **nomevariabile**
- Esempio:

```
int a; //Dichiaro una variabile intera di nome a
```

```
cin >> a; // Quando esegue questa istruzione il programma attende che l'utente scriva qualcosa  
           // da tastiera e schiacci Invio.  
           // Quando viene schiacciato Invio, il programma legge quanto scritto e lo registra (se compatibile) in a/
```

Stampa a video

- `cout << "Messaggio: " << nomevariabile << endl;`
- **`cout << "Il valore in a e`: " << a << endl;`**

Stampa a video la stringa **"Il valore in a e`: 5"** se in a è registrato il valore 5.

`endl` fa andare a capo (endline).

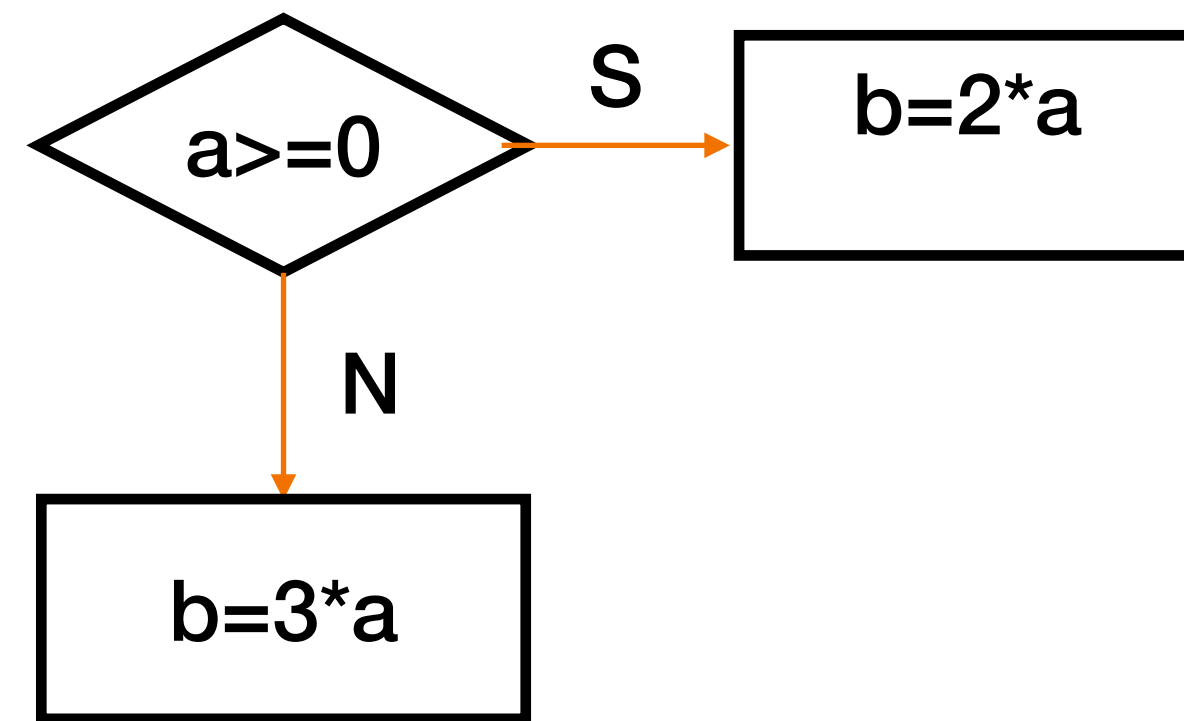
Se voglio comporre messaggi più articolati, posso sbizzarrirmi con la concatenazione di messaggi `<< " ...." <<` e contenuti di variabili

`"Dario e` alto "<< altezza <<" cm" << endl << "Peso (dichiarato): " << peso << " chili" << endl;`

Strutture controllo: selezione

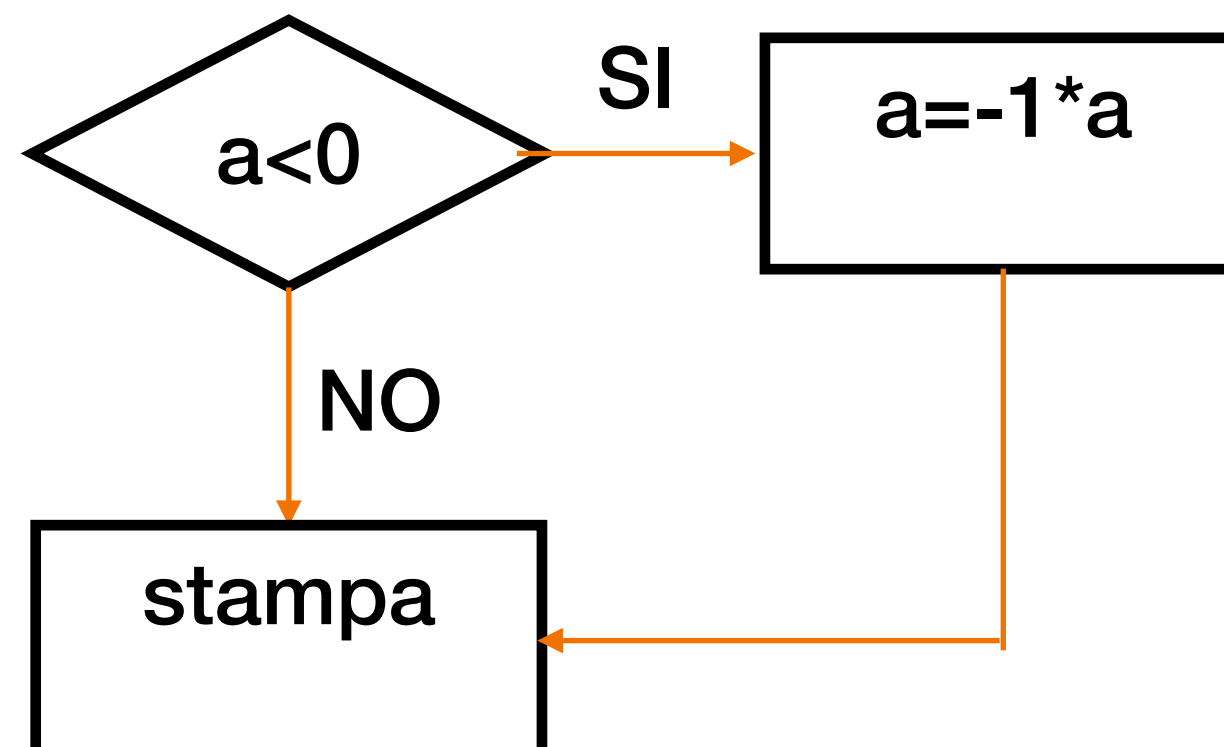
selezione.C

Selezione: **if...else....**



```
if( a >= 0 ){ //Blocco 1
|   b = 2*a;
}
else{ //Blocco 2
|   b = 3*a;
}
```

Selezione "monca": **if...**

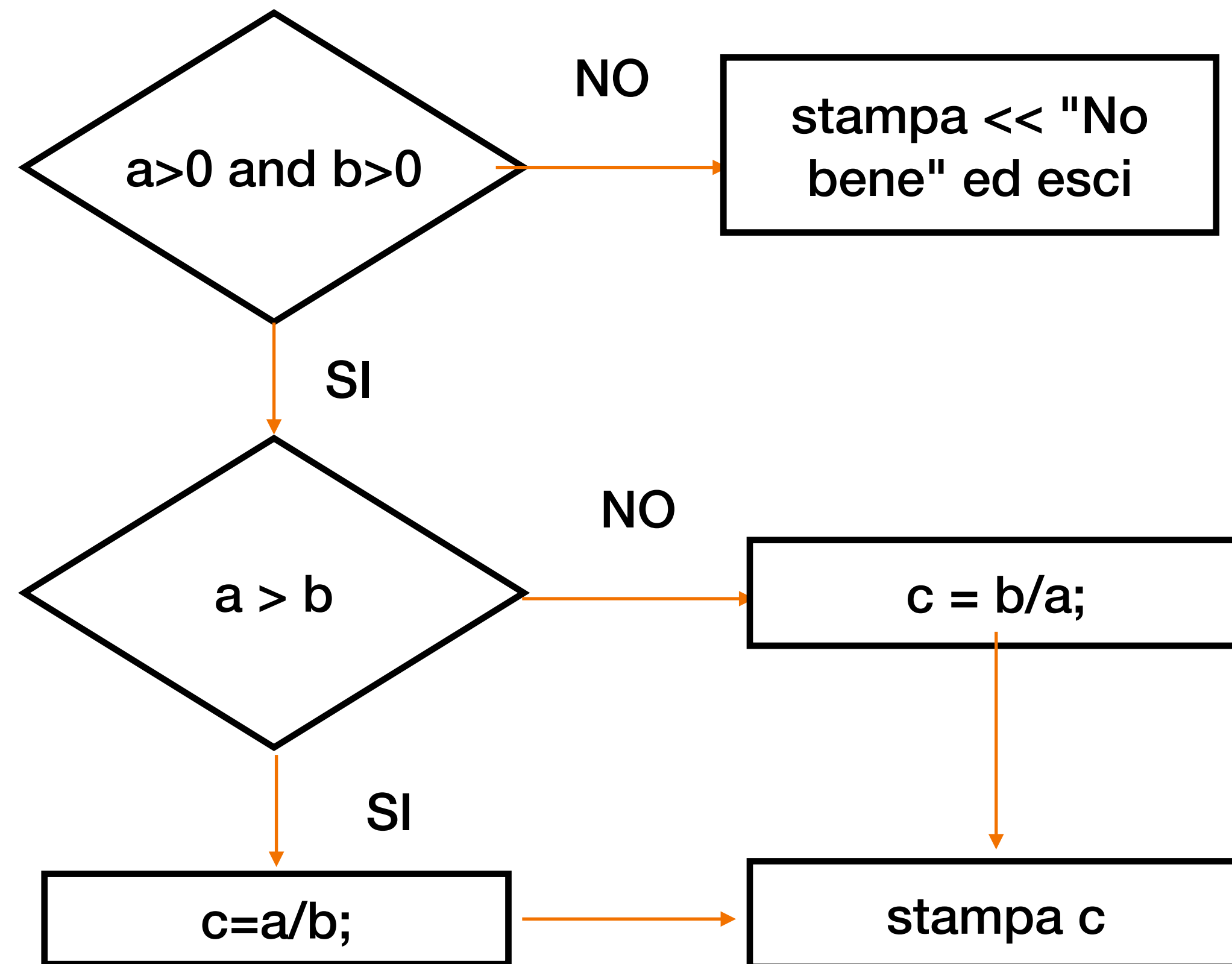


sel_monca.C

```
if( a < 0 ){ //Blocco 1
|   a = -1*a;
}
//No Blocco 2
//Prosegue l'esecuzione

cout << endl << "Risultato, a= " << a << endl;
```

Selezione annidata



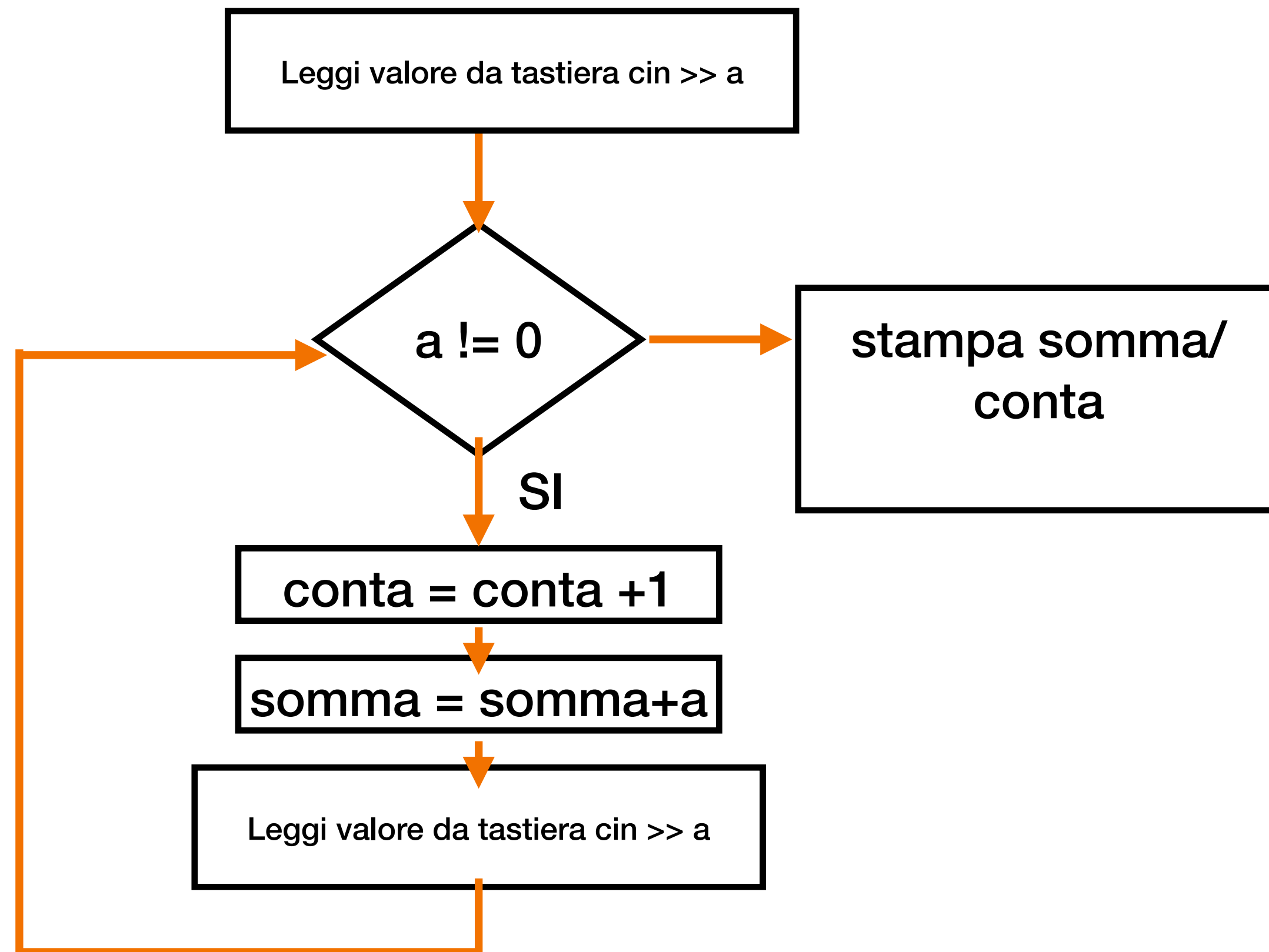
sel_ann.C

```
if (a>0 and b> 0){  
    if (a>=b){  
        c=a/b;  
    }  
    else{  
        c=b/a;  
    }  
    cout << endl << "Risultato, c= " << c << endl;  
}  
else{  
    cout << endl << "No bene! Esco";  
    //L'istruzione return provoca l'uscita dalla funzione main  
    return -1; //Valore "restituito"....ci torniamo  
}
```

Strutture controllo: iterazione

Iterazione (precondizionale): `while()` ...

Il blocco associato al ciclo viene eseguito fino a quando la condizione è VERA



iter_prec.C

```
//Istruzione di lettura da tastiera
cin >> a;

//Il caricamento termina appena viene
  inserito lo 0
while(a != 0){
    conta = conta + 1;
    somma = somma + a;
    cin >> a;
}
```

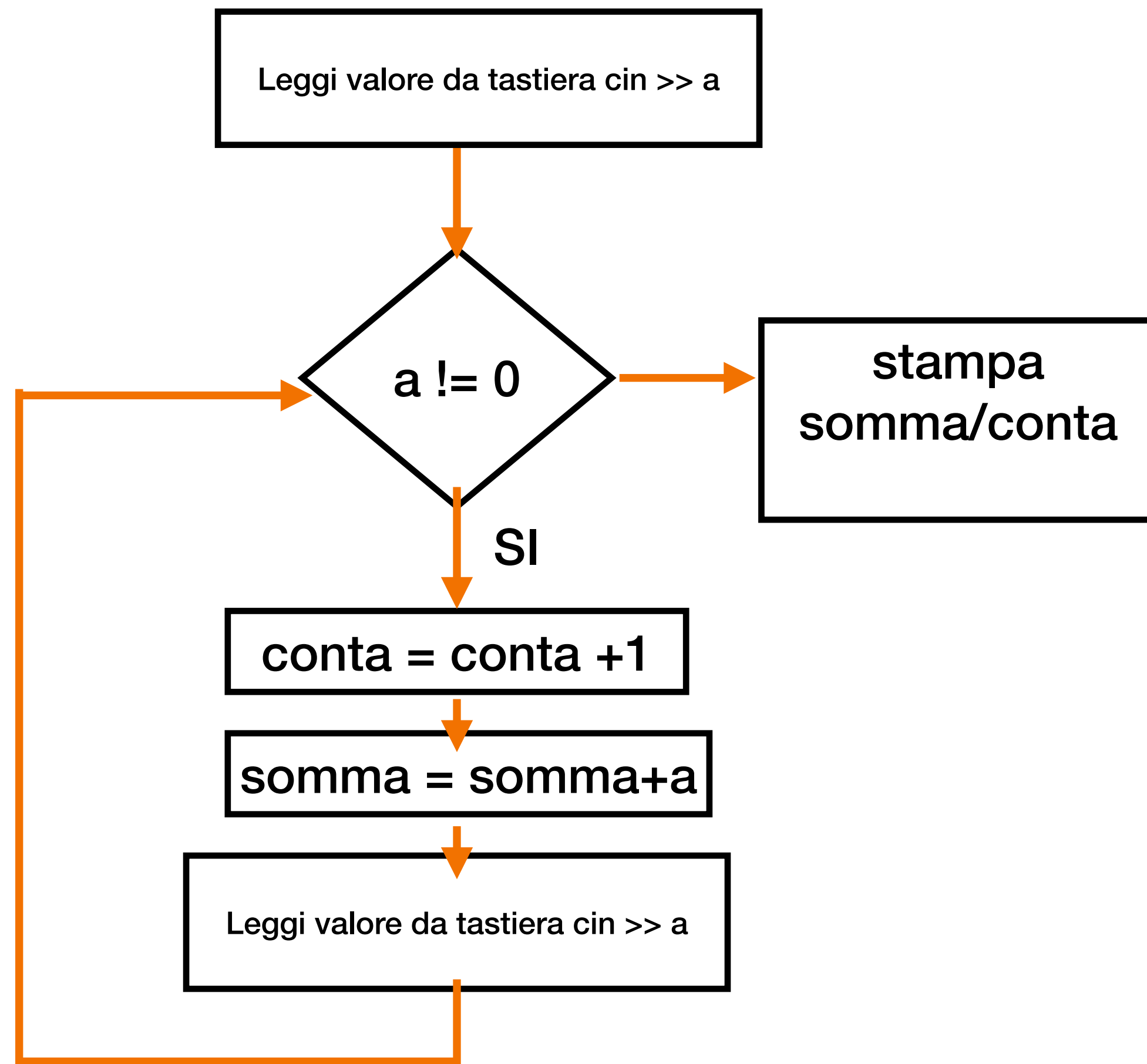
Esistono varianti sul tema:

- iterazione postcondizionale
- for (aka il fatto-apposta per gli array)

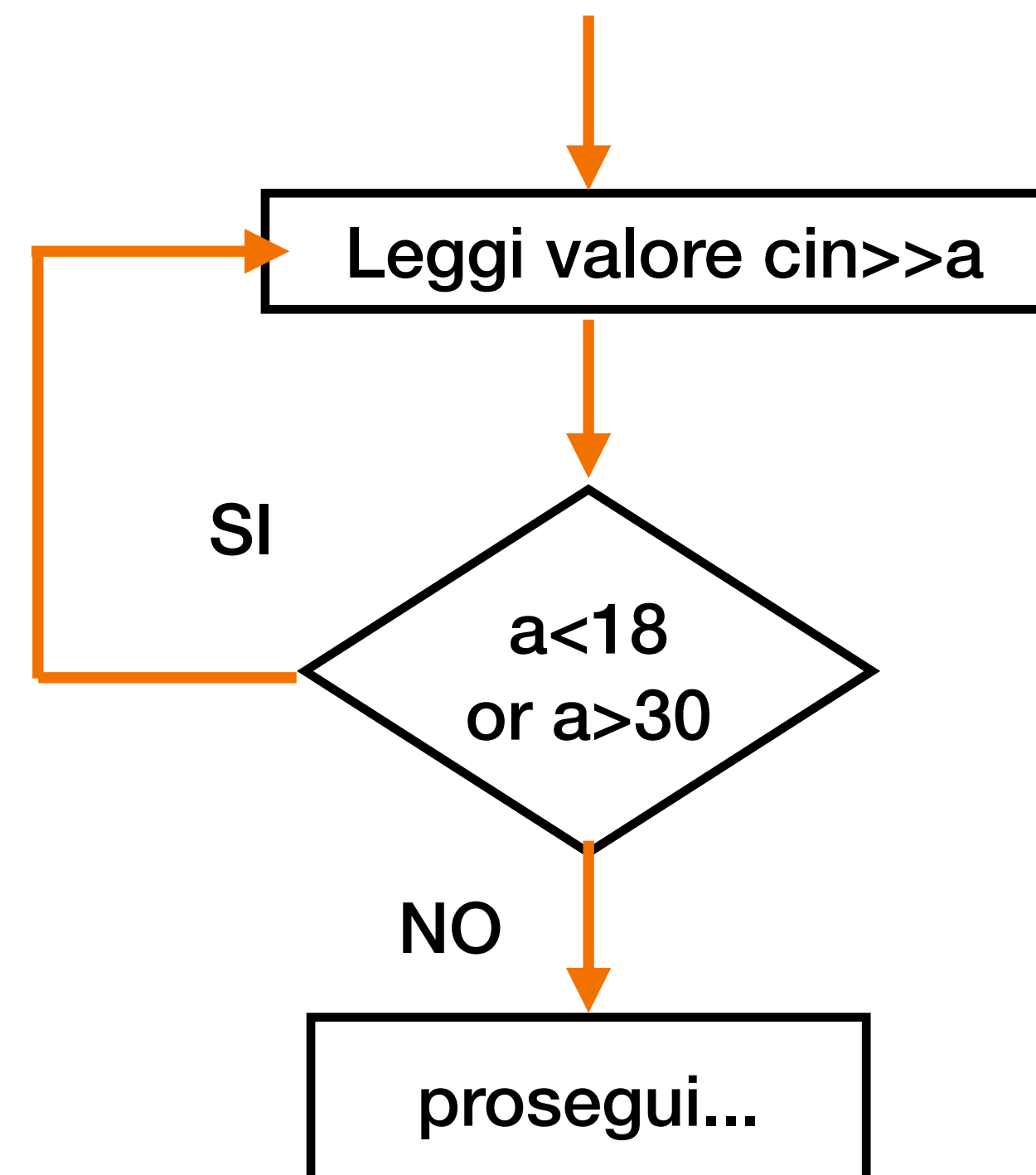
Stay tuned....

Strutture controllo: iterazione

Iterazione (precondizionale): **while(cond)** ...



Iterazione (postcondizionale): **do ...while(cond)**



Osservazione: ne basta una...

Teorema Jacopini-Bohm: basta che linguaggio abbia:

- sequenza
- (uno) selezione,
- (uno) iterazione

Linguaggio Turing completo
(possiamo usarlo per codificare
tutti gli algoritmi)



Quale scegliere?

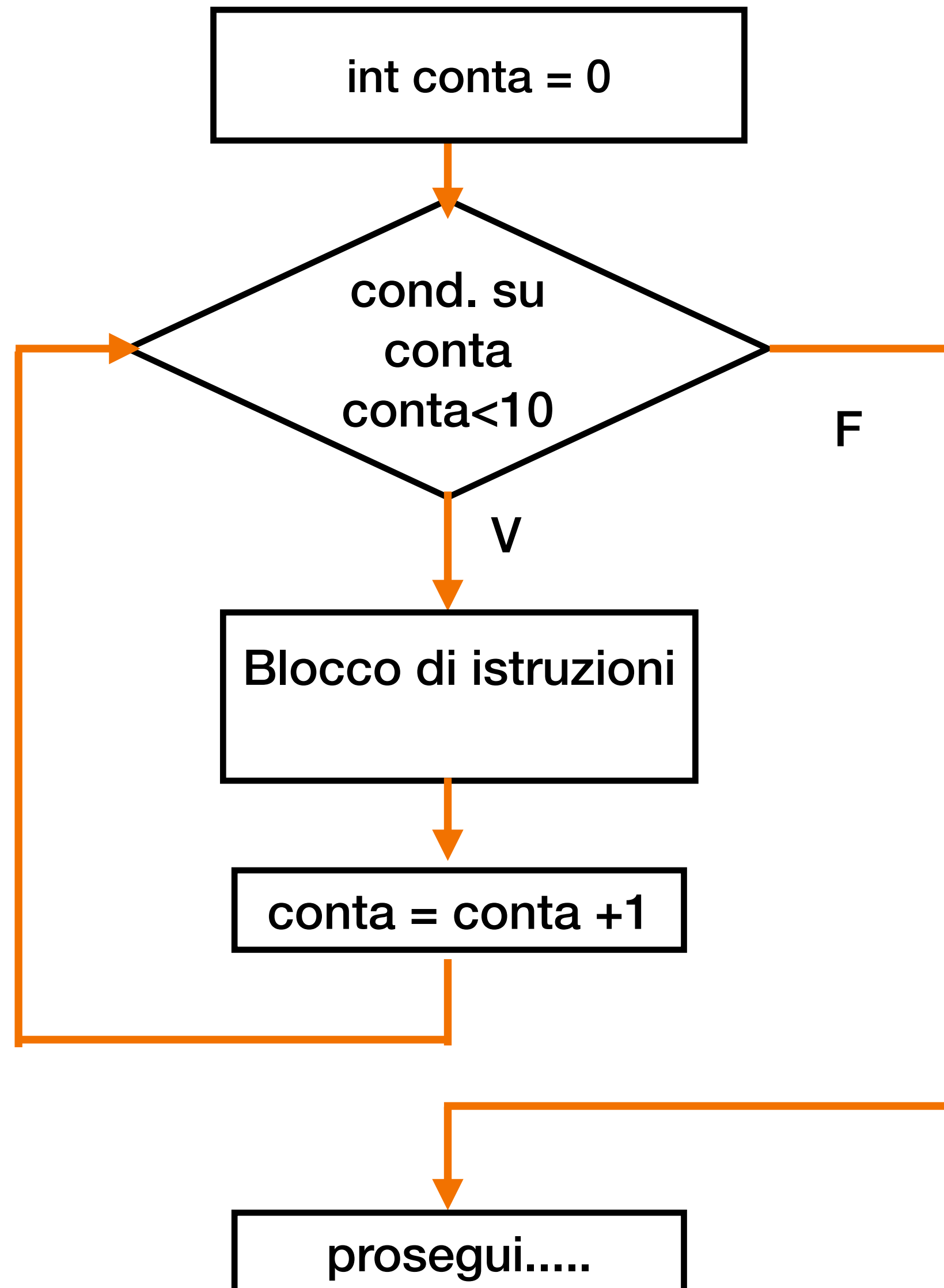
-pre-condizionale: possibili zero esecuzioni del corpo.

-post-condizionale: almeno una esecuzione del corpo.

Dipende dal problema. Il linguaggio viene incontro al programmatore, permettendogli di scegliere il costrutto che meglio si adatta alle caratteristiche del problema.

ESERCIZIO: provate a convertire un while pre-condizionale in un while post-condizionale e viceversa

...e come se non bastasse...



```
for(int conta = 0; conta < 10; conta++){  
    //Blocco di istruzioni  
}
```

In sostanza: while pre-condizionale con uso di contatore inizializzato e incrementato direttamente nel costrutto.

Quale scegliere?

- pre-condizionale: possibili zero esecuzioni del corpo.
- post-condizionale: almeno una esecuzione del corpo.
- for: tipicamente quando il numero di iterazioni è controllato dal valore di un contatore....

